

Digital design verification

Class based environment for ALU

Made by:

Hagar Emadeldin Fathy Emara

Submitted to:

Eng/ Mohamed Ahmed El-adawy

The hierarchy and the flow of the test environment

1. Class transaction: this is the data container. It holds the set of inputs (A, B, Opcode, Cin, rst) and corresponding outputs (Result, Carry..) , and includes a display() function to print values to the console. In addition, inputs tagged with rand so the system generates random values for them.

2. Class generator randomizes the fields inside transaction and sends them through a mailbox (gen2drv_mbx) to the driver. new(mbx) connects this generator to an output mailbox.

3. Class driver: Pulls transactions from the generator's mailbox, waits for a clock edge (negedge clk), and applies those values directly onto the virtual interface pins (intf).

4. Class monitor: listens to the interface pins at every clock cycle, captures the inputs and resulting outputs, packages them into a new transaction, and passes it to the scoreboard.

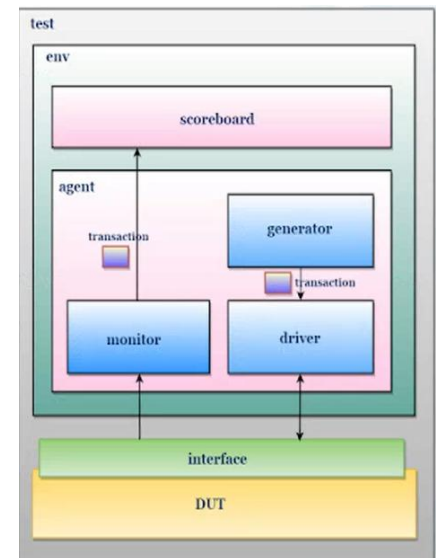
5. Class agent: instantiates all three components and runs their tasks concurrently using fork...join_any. function new(...) instantiates and wires together the generator, driver, and monitor. fork ... join_any starts generator, driver, and monitor tasks running simultaneously in parallel.

6. Class scoreboard reads transactions sent by the monitor, calculates what the answer should be using its own software math logic (exp_result), and compares it against what the ALU hardware actually produced.

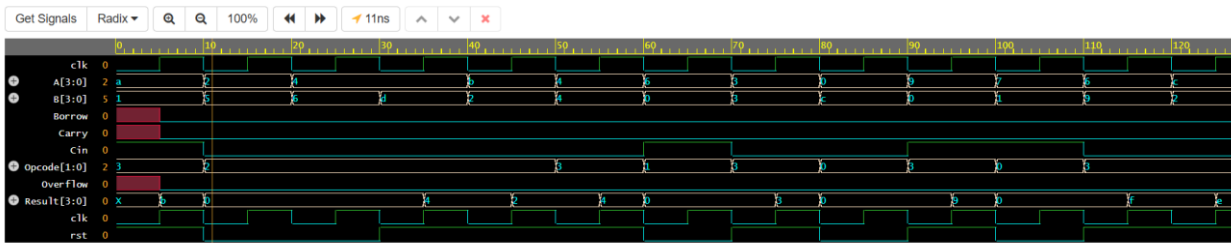
7. Class env connects the agent and scoreboard together via mailboxes and starts them running.

8. module test: initializes the env, configures the total number of transactions (repeat_count = 20).

module tb_top is the top-level test bench wrapper. it generates the clock signal (clk), instantiates the alu hardware and wires it to the alu_intf interface, and triggers the test module and ends the simulation after #300 time units in the implemented code.



The result and the corresponding waveform:



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

As illustrated in the waveform:

When the reset is 0 (active low), the result is 0.

Based on the code: when the opcode was 3, A is a, B is 1, the result is the OR operation of them as implemented in the code, and so on.

```
testbench.vv
202 logic Carry;
203 logic Borrow;
204 logic Overflow;
205 endinterface
206
207 module tb_top;
208 bit clk;
209 alu_intf intf(clk);
210
211 alu DUT (.clk(intf.clk), .rst(intf.rst), .A(intf.A), .B(intf.B),
212 .Cin(intf.Cin), .Opcode(intf.Opcode), .Result(intf.Result),
213 .Carry(intf.Carry), .Borrow(intf.Borrow), .Overflow(intf.Overflow));
214
215 test t1(intf);
216
217 initial begin
218   clk = 0;
219   forever #5 clk = ~clk;
220 end
221 initial begin
222   #300 $finish;
223 end
224 initial begin
225   $dumpfile("dump.vcd");
226   $dumpvars(0, tb_top);
227 end
228 endmodule

design.vv
1 module alu(clk, rst, A, B, Cin, Opcode, Result, Carry, Borrow, Overflow);
2
3 input clk;
4 input rst;
5 input [3:0] A;
6 input [3:0] B;
7 input Cin;
8 output reg [3:0] Result;
9 output reg Carry;
10 output reg Borrow;
11 output reg Overflow;
12
13 always @(posedge clk or negedge rst) begin
14   if (!rst) begin
15     Result <= 4'b0000;
16     Carry <= 1'b0;
17     Borrow <= 1'b0;
18     Overflow <= 1'b0;
19   end else begin
20     Carry <= 1'b0;
21     Borrow <= 1'b0;
22     Overflow <= 1'b0;
23     case (Opcode)
24       2'b00: begin
25         (Carry, Result) <= A + B + Cin;
26         Overflow <= (A[3] == B[3]) && (Result[3] != A[3]);
27       end
28     endcase
29   end
30 end
```

Log Share

Errors: 0, Warnings: 0
End time: 15:42:56 on Jul 21, 2026, Elapsed time: 0:00:02
*** Summary ***
qrun: Errors: 0, Warnings: 0
vlog: Errors: 0, Warnings: 0
vopt: Errors: 0, Warnings: 1
vsim: Errors: 0, Warnings: 0
Totals: Errors: 0, Warnings: 1
Finding VCD file...
./dump.vcd
[2026-07-21 19:42:56 UTC] opening EPWave...
Done